WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Encoders and Decoders

Lecture 08: one-hot codes, encoders, priority encoders and decoders

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 One-hot codes

02 Encoders

03 Priority encoders

04 Decoders

05 Summary

01

One-hot codes

One-hot encoding

Decimal	Binary	One-hot
0	00	0001
1	01	0010
2	10	0100
3	11	1000

Definition (One-hot code)

In an n-bit *one-hot* code, exactly one bit is 1 ('hot') and the other $n - 1$ are 0. The number k has its hot bit k places from the right, at index k.

Example (7 in a 10-bit one-hot code)

Bit 7, counting from bit 0 on the right, is the hot one: `0010000000`.

Binary or one-hot?

Binary

- n bits give 2^n values: compact.
- To ask ‘is the value 5?’, you need a circuit that compares all the bits.

One-hot

- n bits give only n values: many more wires.
- Each bit already answers ‘is the value k?’: just look at bit k.

Circuits often need both, so we need converters:

- an encoder turns one-hot into binary;
- a decoder turns binary into one-hot.

The DX8’s stepper (Lecture 01) is one-hot: it turns on one of its outputs $s_1, \ldots, s_8$ at a time.

02

Encoders

The 4-to-2 encoder

x_3	x_2	x_1	x_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Definition (Encoder)

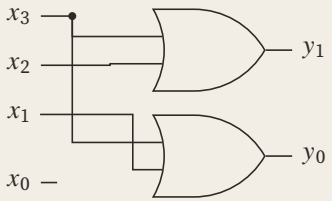
A 2^N-to-N *encoder* turns a 2^N-bit one-hot input into the N-bit binary number of its hot bit.

The book calls it a 4x2 encoder.

Only four inputs are valid, so the table has four rows. Read off each output:

- y_1 is 1 for the values 2 and 3: $y_1 = x_3 + x_2$;
- y_0 is 1 for the values 1 and 3: $y_0 = x_3 + x_1$.

The circuit



- Two OR gates. x_0 is not connected at all: when it is the hot bit, the output is 00 anyway.
- In general, output bit j is the OR of the inputs whose index has a 1 in bit j.

Example (The 8-to-3 encoder)

$y_2 = x_4 + x_5 + x_6 + x_7, \quad y_1 = x_2 + x_3 + x_6 + x_7, \quad y_0 = x_1 + x_3 + x_5 + x_7$

Two problems

input	$y_1 y_0$
0001	00
0000	00
1001	11
0110	11

1. **No hot bit.** Input 0000 gives 00, exactly like the valid input 0001. The output cannot tell us the input was not one-hot.
2. **Several hot bits.** Inputs such as 1001 or 0110 give 11, an answer that appears nowhere in the truth table.

- The book's first fix adds a valid output v: 1 exactly when the input is one-hot,

$$v = x_3 x_2' x_1' x_0' + x_3' x_2 x_1' x_0' + x_3' x_2' x_1 x_0' + x_3' x_2' x_1' x_0.$$

- That flags bad inputs, but still gives no useful answer for them. The priority encoder does.

03

Priority encoders

The priority encoder

x_3	x_2	x_1	x_0	y_1	y_0	v
1	X	X	X	1	1	1
0	1	X	X	1	0	1
0	0	1	X	0	1	1
0	0	0	1	0	0	1
0	0	0	0	0	0	0

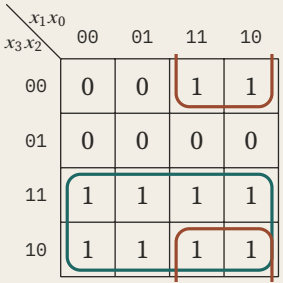
X: either value

Definition (Priority encoder)

A *priority encoder* outputs the index of the highest hot bit, and ignores the bits below it. Its output v is 1 when at least one bit is hot.

Every input now has a meaningful output: 1001 gives 11 because x_3 wins, and 0110 gives 10 because x_2 wins.

Its equations



Karnaugh map of y_0

- y_0 is 1 when the highest hot bit is x_3 or x_1: x_3 or $x_2'x_1$.
- y_1 is 1 when it is x_3 or x_2.

$$y_1 = x_3 + x_2, \qquad y_0 = x_3 + x_2'x_1$$

$$v = x_3 + x_2 + x_1 + x_0$$

The book's circuit computes $y_0 = x_3 + x_3'x_2'x_1$; it is the same function, since $a + a'b = a + b$ (Lecture 03).

Where priority encoders are used

- **Requests with priorities.** Several devices ask for attention at once; the one with the highest priority is served first, and v says whether anyone is asking at all.
- **Finding the first 1.** The index of the highest 1 in a word tells, for example, how far a number must be shifted to put its leading 1 first.

> **Remark**
>
> Logisim-evolution has a ready-made Priority Encoder in the Plexers library. Build your own first, then compare.

04

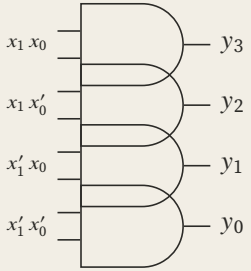
Decoders

The 2-to-4 decoder

Definition (Decoder)

An N-to-2^N *decoder* turns an N-bit binary number into the 2^N-bit one-hot code: the opposite of an encoder.

x_1	x_0	y_3	y_2	y_1	y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



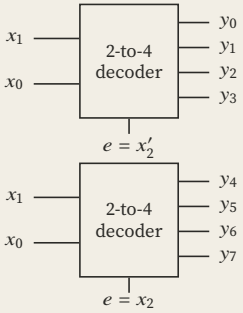
Each output is 1 on one row only, so it is a single minterm, $y_i = m_i$:

$$y_0 = x_1' x_0' \quad y_1 = x_1' x_0$$
$$y_2 = x_1 x_0' \quad y_3 = x_1 x_0$$

The book calls it a 2x4 decoder; its table names the inputs x_1, x_2.

Decoders and demultiplexers

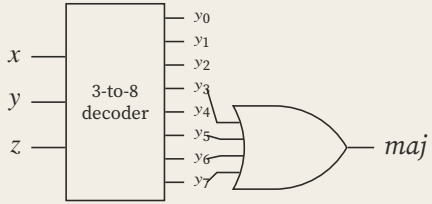
- A decoder with an enable input e: $y_i = e \cdot m_i$. With $e = 0$ all outputs are 0.
- That is exactly the 1-to-4 demultiplexer of Lecture 07, with the data input called e.
- Enables let small decoders build larger ones:



A 3-to-8 decoder from two 2-to-4 decoders: both get x_1x_0, and x_2 picks which one is enabled.

A decoder can compute any function

A decoder's outputs are all the minterms. OR together the ones you need, and you have the canonical sum of products (Lecture 04).



- The majority function: $\sum m(3, 5, 6, 7)$, so $maj = y_3 + y_5 + y_6 + y_7$.
- The sum bit of a 3-bit addition, 1 when an odd number of inputs are 1: $y_1 + y_2 + y_4 + y_7$ (Unit 16).
- One decoder can feed several ORs, one per function.

Decoders in a computer

- **Choosing by address.** The DX8 has 8 registers with 3-bit addresses. A 3-to-8 decoder turns the address into one enable signal per register, so only the addressed register is written `(Unit 19)`.
- **Memory.** The same idea selects one byte out of 256 with an 8-bit address.
- **Instructions.** Decoding an instruction's operation code tells the computer which operation to perform `(Unit 20)`.

> **Remark**
>
> An encoder undoes a decoder: decode a 2-bit number to one-hot, encode it again, and you get the number back.

Try it

1. Which inputs of an 8-to-3 encoder feed y_1?
2. A 4-to-2 priority encoder gets 0110. What are $y_1 y_0$ and v?
3. Write output y_5 of a 3-to-8 decoder with inputs $x_2 x_1 x_0$.
4. How many AND gates does a 4-to-16 decoder need, and with how many inputs?

Try it

1. Which inputs of an 8-to-3 encoder feed y_1?
   x_2, x_3, x_6, x_7: the indices with a 1 in bit 1.
2. A 4-to-2 priority encoder gets 0110. What are $y_1 y_0$ and v?
3. Write output y_5 of a 3-to-8 decoder with inputs $x_2 x_1 x_0$.
4. How many AND gates does a 4-to-16 decoder need, and with how many inputs?

Try it

1. Which inputs of an 8-to-3 encoder feed y_1?
   x_2, x_3, x_6, x_7: the indices with a 1 in bit 1.
2. A 4-to-2 priority encoder gets 0110. What are $y_1 y_0$ and v?
   The highest hot bit is x_2: $y_1 y_0 = 10$, $v = 1$.
3. Write output y_5 of a 3-to-8 decoder with inputs $x_2 x_1 x_0$.

4. How many AND gates does a 4-to-16 decoder need, and with how many inputs?

Try it

1. Which inputs of an 8-to-3 encoder feed y_1?
   x_2, x_3, x_6, x_7: the indices with a 1 in bit 1.
2. A 4-to-2 priority encoder gets 0110. What are $y_1 y_0$ and v?
   The highest hot bit is x_2: $y_1 y_0 = 10$, $v = 1$.
3. Write output y_5 of a 3-to-8 decoder with inputs $x_2 x_1 x_0$.
   $y_5 = x_2 x_1' x_0$: the minterm of 101.
4. How many AND gates does a 4-to-16 decoder need, and with how many inputs?

Try it

1. Which inputs of an 8-to-3 encoder feed y_1?
   x_2, x_3, x_6, x_7: the indices with a 1 in bit 1.
2. A 4-to-2 priority encoder gets 0110. What are $y_1 y_0$ and v?
   The highest hot bit is x_2: $y_1 y_0 = 10$, $v = 1$.
3. Write output y_5 of a 3-to-8 decoder with inputs $x_2 x_1 x_0$.
   $y_5 = x_2 x_1' x_0$: the minterm of 101.
4. How many AND gates does a 4-to-16 decoder need, and with how many inputs?
   Sixteen, one per output, each with four inputs.

05

Summary

Summary

- A one-hot code has exactly one bit set; bit k means the value k.
- A 2^N-to-N encoder turns one-hot into binary: output bit j is the OR of the inputs whose index has a 1 in bit j.
- A plain encoder is wrong for inputs that are not one-hot. A priority encoder reports the highest hot bit, with v saying whether any bit is hot.
- An N-to-2^N decoder turns binary into one-hot: each output is a minterm. With an enable input it is a demultiplexer.
- A decoder and an OR gate compute any function.

Before the next lecture

- Read Sections 4.1.3 and 4.1.4 of the book.
- In Logisim-evolution, build the 4-to-2 priority encoder and the 2-to-4 decoder from gates, and test both on every input.
- Build a 3-to-8 decoder from two 2-to-4 decoders, and use it with an OR gate to compute the majority function.

> **Next lecture** (Unit 9)
>
> Timing of combinational circuits: propagation and contamination delays, and glitches.